HAROKOPIO UNIVERSITY
DEPARTMENT OF INFORMATICS AND TELEMATICS

Cloud Native Applications Project

Architecting a Cloud-Native Application using GitOps and Serverless

Katsimpras Drosos (ais25123)

Harokopio University of Athens
Department of Informatics and Telematics
MSc in Advances in Computer Science and Informatics Systems
Supervisors: Dr. Kousiouris Georgios | Dr. Tsadimas Anargyros | Academic Year: 2025–2026

Table of Contents

Introduction

Where We Start: The Problem

The application was already containerized (Docker) and running in MicroK8s – but locally.

Tied Infrastructure

No independent scaling or remote access.

Manual Configuration

Non-reproducible environment, step-by-step setup.

Manual Deployments

Every code change = manual rebuild/redeploy.

No Secrets Management

Hardcoded credentials, no proper monitoring.

Objective of the Project

Transition to a fully automated, **cloud-native infrastructure on Azure** — without changing the application's underlying business logic.

Project Goals & Methodology

Transforming a local prototype into a resilient cloud-native platform

Multiple Cloud Tech
Each selected for a specific functional role

Resilience Patterns
Claim Check, Retry, Idempotency, Stateless

Serverless Computing
Event-driven logic without permanent compute

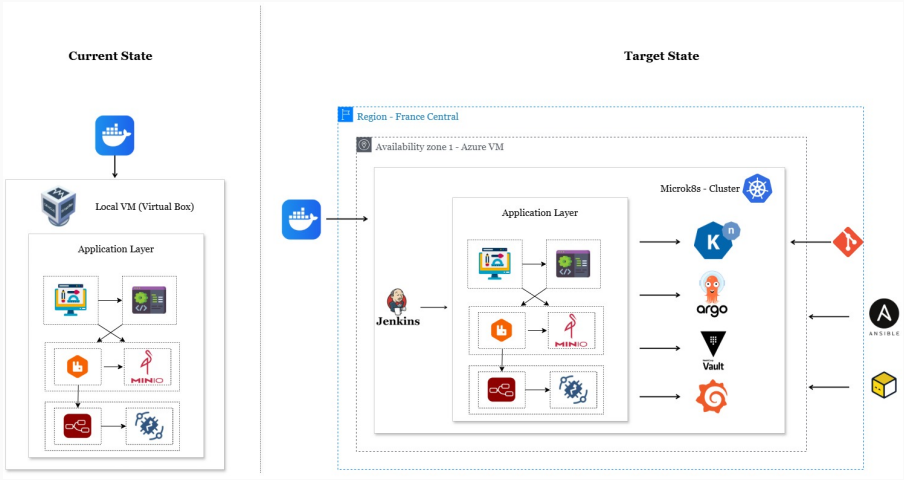
Infrastructure as Code
Full automation via OpenTofu & Ansible

GitOps Practices
Jenkins (CI) + ArgoCD (CD), zero manual steps

Documented SLAs
Based on real load testing measurements

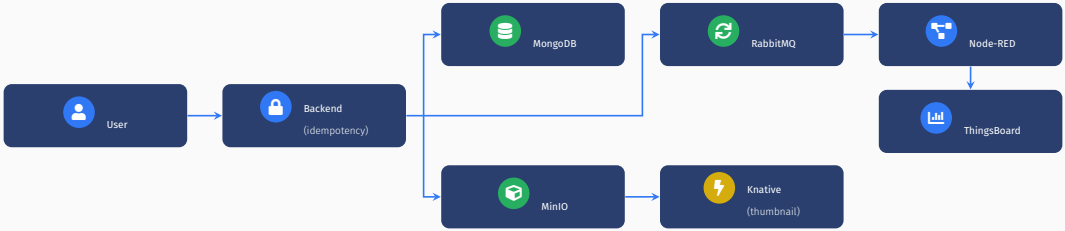
Proposed Architecture & Methodology

The Evolution: Current vs Target State



Smart Warehouse ERP: Data Flow

Orchestrated microservices and the Claim Check pattern



Core Design Decision — Claim Check & Persistent Volumes (PVC):
Images are saved directly to **MinIO**. Only a lightweight URL-reference is passed to RabbitMQ — not the file itself. Saving to MinIO acts as a trigger for the serverless thumbnail generation. Furthermore, PVCs decouple storage from pods, ensuring data survives restarts.

Automation & Security

Infrastructure as Code: OpenTofu & Ansible

OpenTofu — The “Shell”

- Declarative definition of Azure resources in version-controlled files.
- Resource group → VNet/Subnet → NSG → NIC → Linux VM.
- Ubuntu server without UI
- Access only via SSH key — no password.
- Auto-shutdown at 23:00 (FinOps practice).

9 resources · $133.03/month cost estimate (Infracost)

Ansible — The “Inside”

`base-setup` `vault` `monitoring`

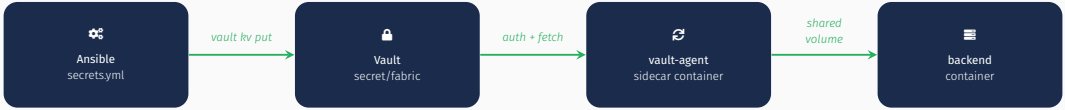
`jenkins` `argocd` `knative` `infracost`

7 independent, **idempotent roles** — safe re-execution after every deallocate/start of the VM, without distinguishing the “first time”.

idempotent tasks: “check if exists” → install only when missing

Secrets Management with HashiCorp Vault

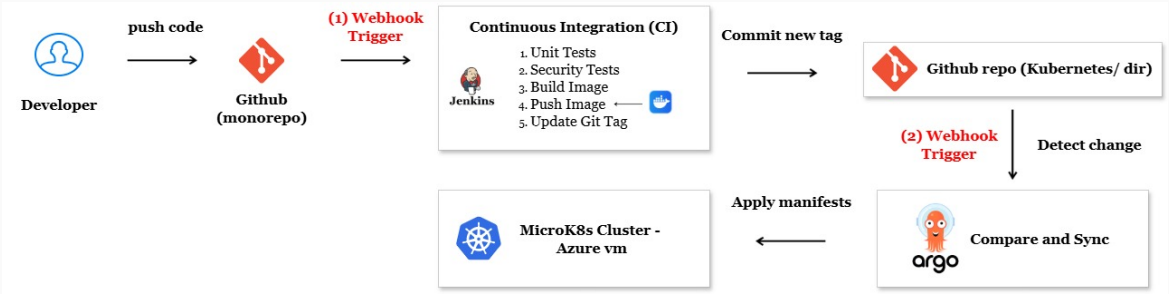
Centralized, secure storage of credentials — no hardcoded values in Git.



- **Kubernetes auth method** — each pod authenticates via its service account, not a static token.
- **Shamir seal** — file storage protected with an unseal key.
- Fallback to environment variables if Vault is unavailable (e.g., local development).

GitOps & CI/CD

GitOps: The End-to-End Pipeline



Selective Execution (Jenkins)

Monorepo + `when { changeset }` → build only when the component changes. No redundant images built.

Self-Heal + Prune (ArgoCD)

Git is the single source of truth. Manual `kubectl` edits are automatically reverted. Resources not in Git are pruned.

Design Patterns & Resilience

Designing for Failure: 4 Resilience Patterns

Claim Check

Large files → MinIO. Only the URL reference travels, keeping message payloads lightweight.

Retry

3 connection attempts with a 2s fixed delay to gracefully handle transient network hiccups.

Idempotency

Client-side key ensures safety. Subsequent requests with the same key → **409 Conflict**.

Stateless

No local session state. Independent instances scale easily using JWT + MongoDB persistence.

Idempotency: Validation in Action

1st Request

POST */api/products*
-F "idempotencyKey=demo-12345"

201 Created

→

2nd Request (same key)

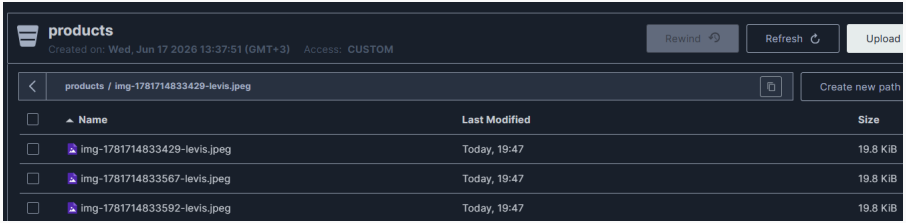
POST */api/products*
-F "idempotencyKey=demo-12345"

409 Conflict
"Duplicate request - already submitted"

Unit Test Coverage: Verified 3 scenarios: new key, reused key, and request without key.

Without Idempotency: The Problem

Scenario: Network retry or accidental double-submit sends the *same* request twice — without a key, the backend has no way to detect this.



Result: Three identical uploads (`img-...429`, `...567`, `...592`) land in MinIO within seconds — duplicate data with no protection at the API layer.

With Idempotency: The Fix in Action

Scenario: Same request, same *idempotencyKey*, sent twice — this time the backend remembers.

Retry Pattern: Implementation

Generic Retry Utility (`retry.js`)

```
async function retryWithDelay(fn, { retries = 3, delayMs = 2000,
                                    label = "operation" } = {}) {
  for (let attempt = 1; attempt <= retries; attempt++) {
    try {
      return await fn();
    } catch (err) {
      if (attempt < retries) {
        await new Promise((resolve) => setTimeout(resolve, delayMs));
      } else {
        throw err;
      }
    }
  }
}
```

Applied to: MongoDB Connection (Startup)

```
retryWithDelay(
  () => mongoose.connect(MONGO_URI)
          .then(() => console.log("MongoDB connected")),
  { retries: 3, delayMs: 2000, label: "MongoDB connection" }
);
```

Serverless Computation

Knative: Event-Driven Thumbnails

Why Knative?

- Runs on existing MicroK8s (no extra infra)
- No vendor lock-in (vs AWS Lambda)
- Scale-to-zero: zero idle resource usage
- Cold start on new events

Triggering Mechanism

1. New file in MinIO bucket
2. MinIO event notification (webhook)
3. Knative function triggered
4. *sharp*: resize 200 × 200 ⟶ *thumb-* prefix

Note: Same design philosophy as the resilience patterns chapter — independent scaling, no permanently reserved resources.

Diagnosis: Infinite Loop in Event-Driven Logic

Symptom

Thumbnail function executes endlessly for the same file.

→

Immediate Action

Removed event notification first — stopped resource drain.

→

Diagnosis

`setObjectTagging()` generates a new event triggering the function again.

→

Root Cause

Guard check compared to string 'true', minio-js returns boolean `true` → silent fail.

The fix (checks both possible formats):

```
if (tags.some(t => t.Key === 'processed' &&
         (t.Value === 'true' || t.Value === true)))
```

Monitoring, Scaling & SLAs

Prometheus + Grafana

CPU & memory per pod, HTTP request count. Custom dashboard for backend + Vault Agent sidecar, plus "Node Exporter Full".

Node-level utilization (24h)

9.4%	49.9%	31.7%
CPU	Memory	Disk

Despite co-hosting Jenkins, ArgoCD, Vault & app.

Pod Autoscaler (HPA)

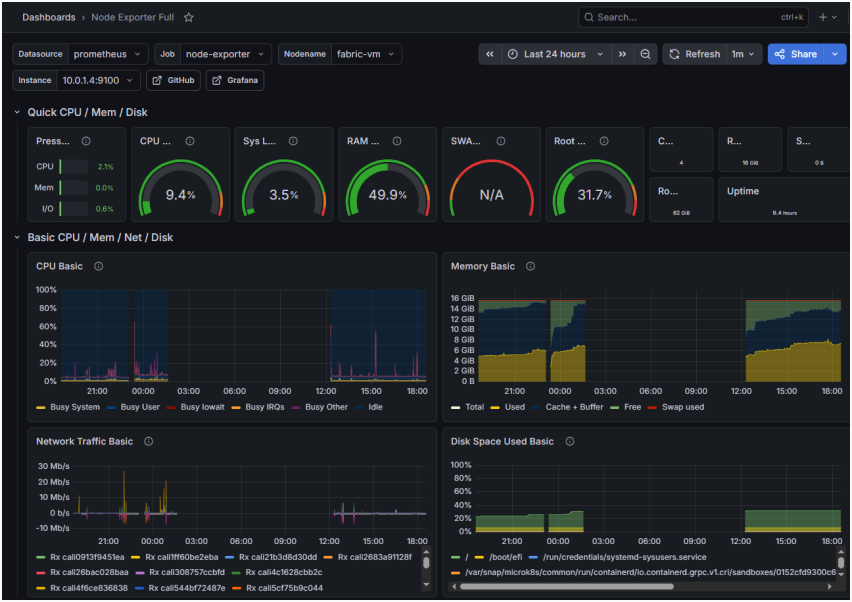
Two independent reference metrics — the higher result between them is adopted:

CPU target	50%
Memory target	70%
min / max replicas	1 → 5

Operates reliably because the backend is stateless — every new replica serves requests immediately.

Grafana: CPU and Memory Usage of the Backend Pod

Grafana: Node Exporter Full Dashboard



From Metrics to Guaranteed SLAs

GET /api/products

100 virtual users · 6,823 requests

14.2ms
p(95)

0.00%
failure rate

POST /api/products

20 virtual users · 1,018 requests

80.1ms
p(95)

0.00%
failure rate

Gold SLA	< 100ms	*GET endpoints (measured 14.2ms)*
Silver SLA	< 250ms	*POST endpoints (measured 80.1ms)*
Bronze SLA	< 500ms	*Worst-case, all requests*

Conclusions & Future Work

Conclusions

- **Cloud-native transformation:** Automated, observable, GitOps-driven platform on Azure.
- **Full GitOps automation:** `git push` flows seamlessly to the cluster via Jenkins & ArgoCD.
- **Resilience by design:** Claim Check, Retry, Idempotency and Stateless patterns.
- **Key Takeaway:** Complexity lives in the *interactions* between technologies, not in isolation.

Known Limitations & Future Work

- **Circuit Breaker:** Not yet implemented (a natural extension to the Retry mechanism).
- **RabbitMQ secrets:** Connection relies on env variables due to a known Docker image limitation.
- **GET pagination:** `/api/products` returns all records (degrades performance under high load).
- **Pod-level FinOps:** Requires additional tooling (e.g., *OpenCost*) beyond *Infracost*.

GitHub Repository

Get the source of the whole project from

github.com/drososkats/fabric-smart-warehouse

Thank you! / Questions? 😊